

14 — Fonctionnement détaillé : Notifications & Box/Conteneurs

Explication du fonctionnement réel (tel que codé), pour répondre à l'oral « comment ça marche ? ».

🔔 Les notifications

Deux canaux partent du **même appel** backend : `SendPushNotification(userID, message)` (dans `API/admin/notifications.go`).

1) L'envoi (backend)

`SendPushNotification` fait **2 choses d'un coup** : - **Notif « cloche » en base** → `bdd.CreateNotification(idUser, message)` insère une ligne dans la table `notification` (`id_user`, `contenu`, `est_lu` = 0). - **Push OneSignal** → un POST à l'API OneSignal ciblant l'utilisateur par son `external_id` (HTTPS/prod uniquement).

Déclencheurs réels :

Événement	Cible	Fichier
Nouvelle inscription	tous les admins (<code>NotifyAllAdmins</code>)	<code>API/admin/users.go</code>
Nouvel article à relire	admins	<code>API/admin/article.go</code>
Événement validé / refusé	le salarié auteur	<code>API/admin/event.go</code>
Article validé	le salarié auteur	<code>API/admin/article.go</code>
Notif manuelle admin (dash)	particuliers / pros	<code>API/admin/notifications.go</code> (<code>SendNotificationToAudience</code>)

`NotifyAllAdmins` récupère les IDs des admins (`GetAllAdminIDs`) et boucle `SendPushNotification` sur chacun.

2) L'affichage (frontend)

Script partagé `Frontend/script/notifCloche.js` (chargé sur admin / client / pro ; `salarie/userInfo.js` pour le salarié). Au chargement : 1. `GET /admin/notifications/user/{userId}` (avec le token) → récupère les notifs. 2. Compte les **non lues** (`est_lu` = 0) → **pastille rouge** sur la cloche s'il y en a. 3. **Clic** sur la cloche → **panneau** listant les notifs + `POST /admin/notifications/user/{userId}/read` → tout passe en lu → la pastille disparaît.

Si la page n'a pas de cloche dans sa nav (admin/pro), le script en **crée une flottante** en haut à droite.

Le push OneSignal

Côté front, le SDK fait `OneSignal.login(userId)` → associe le navigateur à l'ID utilisateur. Le backend cible ce même ID (`external_id`). → notification navigateur même app fermée, **HTTPS uniquement** (désactivé sur localhost).

Endpoints notifs

Méthode	Route	Rôle
POST	/admin/notifications/send	envoi manuel (admin → audience)
GET	/admin/notifications/user/{id}	liste des notifs d'un user
POST	/admin/notifications/user/{id}/read	marquer comme lues

Les conteneurs & box (dépôt / retrait)

Vocabulaire : - **conteneur** (`conteneur` : nom, adresse) = point de collecte physique. - **box** (`box` : numéro, statut, taille) = casier / porte à l'intérieur. - **historique_conteneurs** = table centrale qui trace tout le cycle de vie (réservation → dépôt → retrait).

Le parcours d'un objet vendu, en **3 étapes**, avec 2 codes : un **PIN** (dépôt) et un **code-barres** (retrait).

Étape 1 — Réservation (`bdd.ReserveBox`)

Après l'achat, le système : - cherche un **casier libre** (`statut = 'libre'`), - génère un **PIN** (`code_ouverture`) + un **code-barres** (`code_barre_recuperation = "UC-{annonce}-{box}"`), - crée la ligne `historique_conteneurs` (conteneur, annonce, vendeur, acheteur, codes, `date_reservation`), - annonce → **EN ATTENTE DEPOT**, casier → **reservee**.

→ Le vendeur voit son PIN dans son espace (`GetUserReservations`).

Étape 2 — Dépôt (`bdd.ConfirmDeposit` / `SimulateHardwareDeposit`)

Le vendeur **saisit le PIN** au casier : - vérifie le PIN (`date_depot_effective IS NULL`), - `date_depot_effective = NOW()`, - casier → **occupee**, annonce → **EN BOX** (ou **EN ATTENTE DE RECUPERATION**).

Étape 3 — Retrait (`bdd.CollectObject` / `SimulateHardwareWithdrawal`)

L'acheteur / pro **scanne le code-barres** : - vérifie qu'il a été déposé et pas déjà récupéré, - `date_retrait_effective = NOW()` + `professionnel_id`, - casier → **libre** (code effacé), annonce → **RECUPERE**, - **ajoute l'Upcycling Score** (`CalculateAndAddScore` : poids × coefficient).

Le simulateur (`Frontend/simulateur.html`)

Pas de vraie serrure connectée → la page appelle `/api/hardware/simulate-deposit` (PIN) et `/api/hardware/simulate-withdrawal` (code-barres) pour **imiter le matériel physique**.

Côté admin (`/admin/conteneurs`)

Créer un conteneur, ajouter un casier, changer le statut d'un casier, générer le **rapport logistique PDF** (jsPDF).

Nettoyage auto (`GarbageCollectBox`)

Les casiers réservés > **2 jours sans dépôt** sont libérés automatiquement.

Endpoints box

Méthode	Route	Rôle
GET	/api/admin/conteneurs	liste des conteneurs
GET	/api/admin/conteneur/{id}/boxes	casiers d'un conteneur
POST	/api/admin/conteneur/create	créer un conteneur
POST	/api/admin/box/add	ajouter un casier
PUT	/api/admin/box/update	changer le statut
POST	/api/box/reserve /deposit /collect	flux réservation/dépôt/retrait
POST	/api/hardware/simulate-deposit /simulate-withdrawal	simulateur matériel
GET	/api/user/boxes /api/user/pickups/{id}	dépôts/retraits d'un user

Les statuts (à retenir)

- **Box** : libre → reservee → occupee → libre (après retrait).
- **Annonce (statut_vente)** : EN VENTE → EN ATTENTE DEPOT → EN BOX → RECUPERE .

Phrases à dire à l'oral

- « Chaque action déclencheuse appelle **une seule fonction** qui écrit la notif en base **et** envoie le push — donc la cloche et le push sont toujours cohérents. »
- « Le cycle d'un objet est tracé dans **une table pivot** (historique_conteneurs) avec un **PIN** pour déposer et un **code-barres** pour récupérer ; les statuts du casier et de l'annonce évoluent à chaque étape. »
- « Comme on n'a pas de serrure connectée réelle, un **simulateur** reproduit le comportement du matériel via deux endpoints. »